COMMAND PIPING TOOL

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in The Department of Computer Science and Engineering

Operating Systems and System Programming (25CS2104E)

Submitted by

2510030145 : IVATURI NAGA MOHITH

2510030115 : PEDDI INDRA KIRAN REDDY

2510030200 : CH HARSHITH

Under the guidance of

Dr. A. Seenu



Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCTOBER - 2026.

Abstract

Every Unix shell lets a user connect two programs with the pipe symbol, so that the output of the first program becomes the input of the second. Although this feature is used daily, the mechanism behind it is hidden from the user. This project, the Command Piping Tool, rebuilds that mechanism from first principles in the C programming language using POSIX system calls on Linux. The tool accepts two commands from the user, creates an anonymous pipe with pipe(), creates two child processes with fork(), rewires the standard output of the first child and the standard input of the second child to the two ends of the pipe with dup2(), replaces each child with the requested program using execvp(), and finally synchronises with both children through waitpid().

The design separates the work into three roles: a parent process that orchestrates, a producer child that writes into the pipe, and a consumer child that reads from it. Careful closing of unused file descriptors guarantees that the consumer receives end-of-file and that no process blocks forever. The implementation was validated with ten pipelines covering filtering, counting, sorting, transformation, large data transfer, invalid commands and early termination of the consumer. In every valid case the output of the tool was identical to the output of the same pipeline in the Bash shell. A throughput benchmark from 1 MB to 256 MB showed that the tool moves data at about 2.6 GB/s for large volumes, on par with Bash, and a controlled experiment confirmed that leaving a single write end open makes the pipeline hang. Testing also revealed that a child killed by SIGPIPE was being reported with status 0; the status reporting was corrected using WIFSIGNALED(). The project demonstrates, in a small and readable program, the core operating-system concepts of process creation, inter-process communication, file-descriptor management, program execution and process synchronisation.

Keywords: *Operating systems, inter-process communication, anonymous pipe, fork, dup2, execvp, waitpid, file descriptors, SIGPIPE, Linux system programming.*

List of Figures

Fig. 1. Project development phases....10
Fig. 2. User interaction diagram....11
Fig. 3. System architecture....11
Fig. 4. Kernel pipe buffer....12
Fig. 5. Life cycle of a child process....13
Fig. 6. Flowchart of the implementation....15
Fig. 7. File-descriptor tables before and after redirection....16
Fig. 8. Process interaction timeline....17
Fig. 9. Effect of an unclosed write end....18
Fig. 10. Data-flow trace of echo | tr....22
Fig. 11. Compilation and T1 (ls -l | grep .c)....23
Fig. 12. T2 (cat /etc/passwd | wc -l)....23
Fig. 13. T3 (echo | tr a-z A-Z)....24
Fig. 14. T4: invalid first command....24
Fig. 15. Reference output in Bash....25
Fig. 16. T5 (ls | sort -r)....25
Fig. 17. T6 (seq | tail)....26
Fig. 18. T7: 1 MiB transfer....26
Fig. 19. T8 (printf | sort -u)....27
Fig. 20. T9: invalid second command....27
Fig. 21. T10: early exit and SIGPIPE....28
Fig. 22. Throughput benchmark....29
Fig. 23. Execution time versus data volume....29
Fig. 24. Descriptor-leak experiment....30

List of Tables

Table I. Comparison of inter-process communication mechanisms....9
Table II. Functional requirements....10
Table III. Non-functional requirements....10
Table IV. System calls used in the Command Piping Tool....12
Table V. Pipe parameters measured on the test system....13
Table VI. Hardware and software environment....19
Table VII. Core test cases and expected output....21

Table VIII. Extended test cases.....21
Table IX. Results of all test cases compared with the Bash shell.....28
Table X. Benchmark results (median of five runs).....28
Table XI. Comparison of the Command Piping Tool with the Bash shell pipeline.....30
Table XII. Challenges faced and solutions adopted.....31
Table XIII. Learning outcomes mapped to operating-system concepts.....31

Table of Contents

Abstract....2
List of Figures....3
List of Tables....3
1. Introduction....7
a. Background....7
b. Motivation....7
c. Problem Statement....7
d. Objectives....8
e. Scope of the Project....8
f. Organisation of the Report....8
2. Literature Review....8
a. Inter-Process Communication Mechanisms....8
b. Anonymous Pipes and Named Pipes....9
c. How Shells Implement Pipelines....9
d. Summary of the Review....9
3. Methodology....9
a. Requirements Analysis....10
b. User Interaction Design....10
c. System Architecture....11
d. Core System Calls....12
e. Pipe Internals in the Linux Kernel....12
f. Process Life Cycle....13
g. Algorithm....13
h. Command Parsing....15
i. I/O Redirection and File-Descriptor Logic....16
j. Process Synchronisation and Correctness....17
k. Error Handling and Status Reporting....18
4. Experiments....18
a. Experimental Setup....18
b. Implementation....19
c. Core Test Cases....21

d. Extended Test Cases.....21
e. Performance Benchmark.....22
f. Descriptor-Leak Experiment.....22
g. Worked Trace of the Transformation Test.....22
5. Results.....22
a. Results of the Core Tests.....23
b. Results of the Extended Tests.....25
c. Performance Results.....28
d. Result of the Descriptor-Leak Experiment.....29
e. Discussion.....30
f. Challenges Faced.....30
g. Learning Outcomes.....31
6. Conclusion and Future Work.....31
References.....32

Command Piping Tool: Shell-Style Inter-Process Communication Using Anonymous Pipes in C

1. Introduction

An operating system does far more than run one program at a time. It lets many programs run together and, just as importantly, lets them cooperate. One of the oldest and most elegant forms of cooperation in Unix-like systems is the pipeline. When a user types ls -l | grep .c in a terminal, the shell starts two programs at once and connects the standard output of ls directly to the standard input of grep. Neither program knows that the other exists; each simply writes to or reads from its standard streams, and the operating system moves the bytes between them.

Behind the single pipe character lies a sequence of carefully ordered system calls. The shell must ask the kernel for a pipe, create new processes, redirect file descriptors so that the right process writes to the right end, replace each process with the requested program, and finally wait for the programs to finish. If any one of these steps is done in the wrong order, or if a single unused descriptor is left open, the pipeline can hang forever or lose data. For students of operating systems, this makes the pipeline an ideal case study: it is small enough to implement in one file, yet it touches process management, inter-process communication (IPC), file-descriptor tables, signals and synchronisation all at once [1], [2].

a. Background

The pipe was proposed by Doug McIlroy at Bell Labs, who argued in an internal memo as early as 1964 that programs should be connected "like garden hose". Ken Thompson added the pipe() system call and the | notation to Unix in 1973, in what became Version 3 Unix, and the idea quickly changed how Unix programs were written [6], [7]. Instead of building large programs that do everything, developers wrote small tools such as grep, sort, wc, tr and head, each doing one job well and each reading standard input and writing standard output. Complex tasks are then solved by combining these tools on the command line. This style, known as the Unix philosophy, is still at the heart of Linux, macOS and every POSIX shell today [14].

Inter-process communication is required because every process in a modern operating system runs in its own protected address space. One process cannot simply read a variable belonging to another. The kernel therefore provides explicit channels such as pipes, FIFOs, message queues, shared memory and sockets [3]. Among these, the anonymous pipe is the simplest: it is a one-way byte stream held in a kernel buffer, it has no name in the file system, and it can be shared only between related processes that inherit its descriptors through fork(). These properties match the needs of a shell pipeline exactly.

b. Motivation

Most students use pipelines long before they understand them. Writing a pipeline from scratch exposes several ideas that are easy to state but hard to appreciate: that fork() copies the descriptor table, that execvp() keeps open descriptors while replacing everything else, that end-of-file on a pipe depends on every write end being closed, and that a process killed by a signal reports its status differently from one that exits normally. The motivation for this project was to turn these textbook statements into observable behaviour, by building a tool that prints each step it takes and by running experiments that deliberately break the rules.

c. Problem Statement

Shell-style piping connects the output of one process to the input of another. Reproducing the same behaviour in a C program is not automatic: the programmer must explicitly create the communication channel, manage file descriptors, create and replace

processes, and synchronise their termination. The problem addressed by this project is to design and implement a C-based tool that performs this complete sequence for two user-supplied commands, prints the result of the second command, and makes every internal step visible so that the underlying operating-system concepts can be understood and demonstrated.

d. Objectives

- To create an anonymous pipe with pipe() and understand its read end and write end.
- To create two child processes with fork() and assign them the roles of producer and consumer.
- To redirect standard output and standard input with dup2() so that Command 1 writes into the pipe and Command 2 reads from it.
- To execute arbitrary Linux commands inside the children with execvp().
- To close unused descriptors correctly and synchronise the parent with both children using waitpid().
- To validate the tool with filtering, counting, sorting, transformation, large-data and error-handling tests and compare the results with the Bash shell.
- To measure the throughput of the tool and demonstrate experimentally what happens when a descriptor is left open.

e. Scope of the Project

The scope of the project is a two-command pipeline implemented with POSIX system calls on Linux. The user enters Command 1 and Command 2, each with its own arguments; quoted arguments such as "hello operating system" are supported. Features of a full shell such as pipelines of more than two stages, file redirection with < and >, background jobs, environment-variable expansion and built-in commands are outside the scope of this work and are discussed as future enhancements.

f. Organisation of the Report

Section 2 reviews inter-process communication mechanisms and how existing shells implement pipelines. Section 3 describes the methodology: requirements, architecture, system calls, pipe internals, the algorithm, descriptor logic, synchronisation and error handling. Section 4 presents the experimental setup, the implementation and the test plan. Section 5 reports the results of the functional tests, the benchmark and the descriptor-leak experiment, and discusses the challenges faced. Section 6 concludes the report and outlines future work.

2. LITERATURE REVIEW

This section places the project in the context of the IPC facilities offered by Unix-like operating systems and of the way real shells build pipelines.

a. Inter-Process Communication Mechanisms

POSIX systems offer several IPC mechanisms, each suited to a different pattern of communication [2], [3]. Table I compares the most common ones. Pipes and FIFOs carry an unstructured byte stream in one direction. Message queues preserve message boundaries. Shared memory is the fastest because data is not copied through the kernel, but it requires separate synchronisation such as semaphores. Unix domain sockets support two-way communication and are used by many local services. Signals carry only a small number identifying an event, but they play an important role in pipelines, as SIGPIPE shows later in this report.

TABLE I. COMPARISON OF INTER-PROCESS COMMUNICATION MECHANISMS

Mechanism	Direction	Related processes only?	Name in file system	Typical use
Anonymous pipe	One-way	Yes (via fork)	No	Shell pipelines
Named pipe (FIFO)	One-way	No	Yes (mkfifo)	Unrelated local processes
Message queue	Message based	No	Key or name	Structured messages
Shared memory	Both ways	No	Key or name	Fast bulk data, needs locking
Unix domain socket	Both ways	No	Optional path	Local client-server
Signal	Notification only	No	No	Events such as SIGINT, SIGPIPE

For a shell pipeline the anonymous pipe is the natural choice. The two commands are always started by the same parent, so they are related processes; the data flows in one direction only; and neither program needs to know that it is talking to another program, because the pipe looks exactly like an ordinary file descriptor.

b. Anonymous Pipes and Named Pipes

An anonymous pipe is created with pipe() and exists only as two file descriptors. It disappears automatically when the last descriptor referring to it is closed. A named pipe, or FIFO, is created with mkfifo() and appears as a special file in the file system, so unrelated processes can open it by name. Internally both use the same kernel pipe buffer and the same blocking rules. Because the Command Piping Tool starts both commands itself, an anonymous pipe is sufficient and avoids leaving any files behind.

c. How Shells Implement Pipelines

Every POSIX shell follows the same basic recipe that this project implements: create a pipe, fork one child per command, use dup2() in each child to attach the pipe to standard input or output, close the unused descriptors, call a member of the exec family, and wait in the parent [1], [2]. Bash generalises this to N commands by creating N-1 pipes and also places all the children in one process group so that job control and Ctrl+C affect the whole pipeline. The exit status of a pipeline in Bash is normally the status of its last command, with the PIPESTATUS array holding the status of every stage.

The xv6 teaching operating system developed at MIT contains a shell of only a few hundred lines whose pipe case is almost identical to the core of this project: it calls pipe(), forks a left and a right child, closes descriptor 1 or 0 and duplicates the pipe end into the freed slot, and then waits for both children [11]. The C library function popen() offers a simpler interface for running a single command with a pipe to or from the calling program, but it hides the steps that this project sets out to show. Studying these implementations confirmed that the design chosen for the tool matches established practice.

d. Summary of the Review

The review shows that the anonymous pipe is the standard and most suitable mechanism for shell-style command chaining, that the sequence pipe, fork, dup2, close, exec and wait is used by every major shell, and that correct handling of descriptors and exit statuses is the main source of subtle bugs. These findings directly shaped the requirements and design in Section 3.

3. METHODOLOGY

The tool was developed in six phases, shown in Fig. 1: study of the required system calls, design of the process and descriptor layout, implementation in C, functional testing, evaluation against Bash, and documentation. This section explains the decisions taken in each phase.

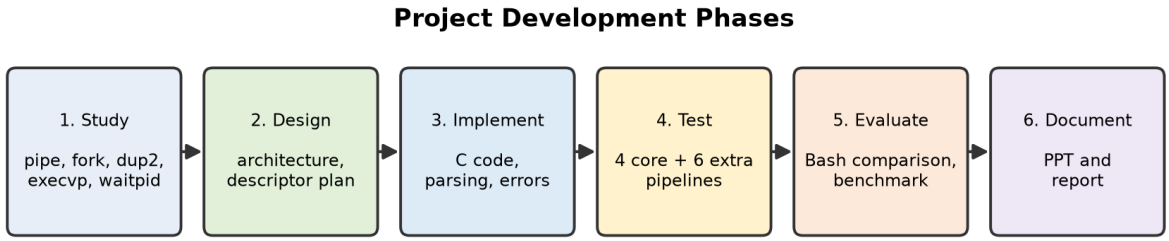


Fig. 1. Development phases followed in the project.

a. Requirements Analysis

The functional requirements of the tool are listed in Table II and the non-functional requirements in Table III. FR7 was added during testing, when it became clear that a child killed by a signal must be reported differently from one that exits normally.

TABLE II. FUNCTIONAL REQUIREMENTS

ID	Requirement
FR1	The tool shall prompt the user for Command 1 and Command 2 and accept arguments for each.
FR2	The tool shall keep text inside single or double quotes together as one argument.
FR3	The tool shall create an anonymous pipe and two child processes.
FR4	Command 1's standard output shall be redirected into the pipe and Command 2's standard input shall be read from it.
FR5	The output of Command 2 shall appear on the terminal.
FR6	The tool shall print each internal step (pipe, fork, redirection, execution).
FR7	The tool shall report how each child finished: its exit status or the signal that terminated it.
FR8	The tool shall report an error if a command cannot be executed, without hanging.

TABLE III. NON-FUNCTIONAL REQUIREMENTS

Attribute	Requirement
Correctness	Output must be identical to the same pipeline run in Bash.
Reliability	No process may block forever; no zombie processes may remain.
Performance	Throughput must be comparable to Bash for large data volumes.
Portability	Only POSIX system calls; builds with GCC or Clang on any Linux distribution.
Readability	Single source file of about 120 lines, commented for teaching use.
Usability	Plain prompts and clear error messages; no special syntax to learn.

b. User Interaction Design

The tool is used from a terminal. When started, it shows a banner and two prompts, Command 1 and Command 2. After both lines are entered, it prints each step it performs, then the output of Command 2 under the heading Final output, and finally one line per child describing how it finished. The interactions available to the user are summarised in Fig. 2.

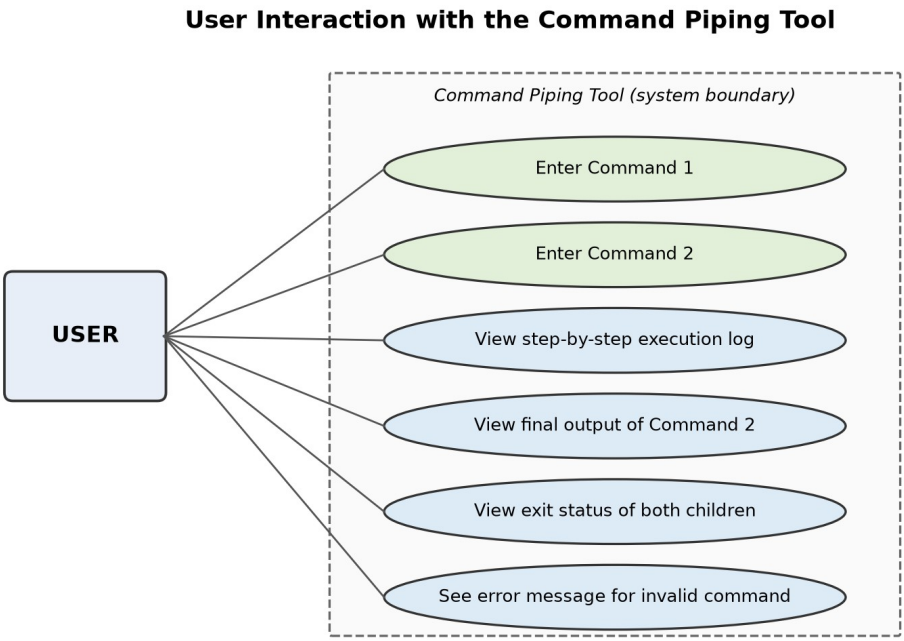


Fig. 2. User interaction (use-case) diagram of the Command Piping Tool.

c. System Architecture

The architecture follows the same structure used inside real shells. Three processes take part, as shown in Fig. 3. The parent process reads the two commands from the user, creates the pipe and creates both children. Child 1, the producer, runs Command 1 with its standard output connected to the write end of the pipe. Child 2, the consumer, runs Command 2 with its standard input connected to the read end. The anonymous pipe lives inside the kernel and carries bytes from the producer to the consumer. The output of Command 2 is written to the terminal, which the user sees as the final result.

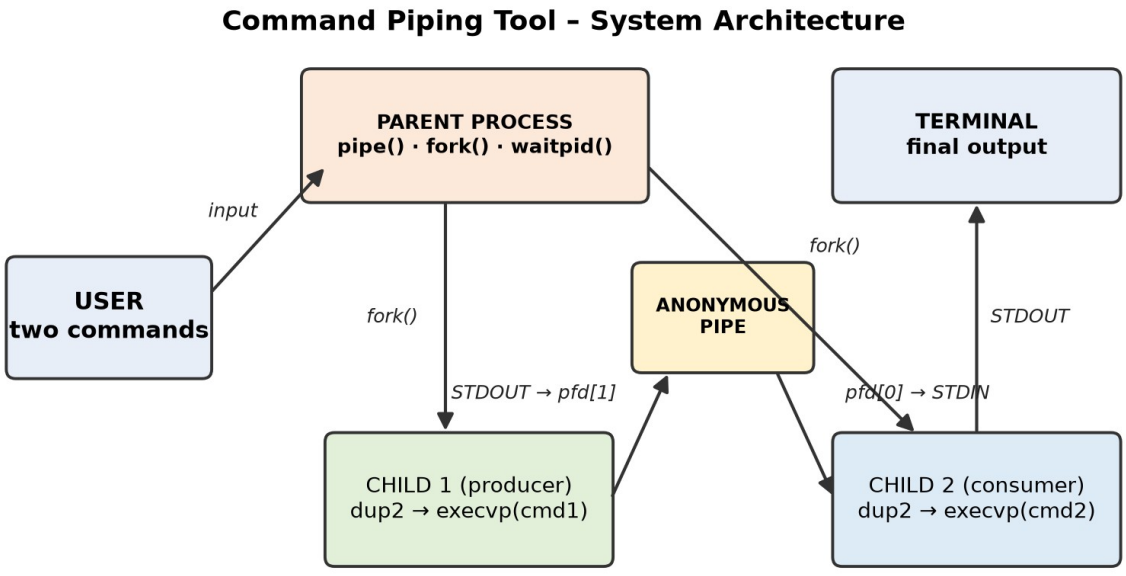


Fig. 3. System architecture showing the parent, the two child processes and the anonymous pipe.

d. Core System Calls

Table IV lists the system calls and macros on which the tool is built. All of them are defined by the POSIX standard [8], so the program is portable to any Unix-like operating system. The behaviour of each call was studied from the Linux manual pages [9] before implementation.

TABLE IV. SYSTEM CALLS USED IN THE COMMAND PIPING TOOL

System call	Header	Purpose in this project	Return value used
pipe(pfd)	<unistd.h>	Creates the anonymous pipe; pfd[0] is the read end and pfd[1] the write end	0 on success, -1 on error
fork()	<unistd.h>	Creates Child 1 (producer) and Child 2 (consumer)	0 in child, child PID in parent, -1 on error
dup2(old, new)	<unistd.h>	Copies a pipe end onto STDOUT_FILENO or STDIN_FILENO	New descriptor, -1 on error
close(fd)	<unistd.h>	Releases unused pipe ends so that EOF is delivered	0 on success
execvp(file, argv)	<unistd.h>	Replaces the child image with the requested command, searching PATH	Returns only on failure (-1)
waitpid(pid, &st, 0)	<sys/wait.h>	Parent waits for each child and collects its status	PID of the terminated child
WIFEXITED / WIFSIGNALED	<sys/wait.h>	Decode whether a child exited normally or was killed by a signal	Non-zero if true
perror(msg)	<stdio.h>	Prints a readable error message based on errno	-

The pipe() call fills a two-element integer array: pfd[0] refers to the read end and pfd[1] to the write end. The fork() call duplicates the calling process; the child receives a copy of the parent's file-descriptor table, which is why both children can reach the pipe that the parent created. The dup2() call makes one descriptor number refer to the same open file as another, and execvp() loads a new program into the current process while keeping its descriptor table. This last property is what makes redirection work: the descriptors are arranged before execvp(), and the new program simply inherits them.

e. Pipe Internals in the Linux Kernel

On Linux a pipe is implemented by the pipe file system as a small ring of page-sized buffers [12], [13]. By default the ring has 16 slots of 4 KiB, giving a capacity of 64 KiB, as illustrated in Fig. 4. The writer adds data at the head of the ring and the reader removes it from the tail. The values measured on the test system are listed in Table V; the capacity of 65,536 bytes was confirmed both with fcntl(F_GETPIPE_SZ) and by writing single bytes in non-blocking mode until the kernel returned EAGAIN.

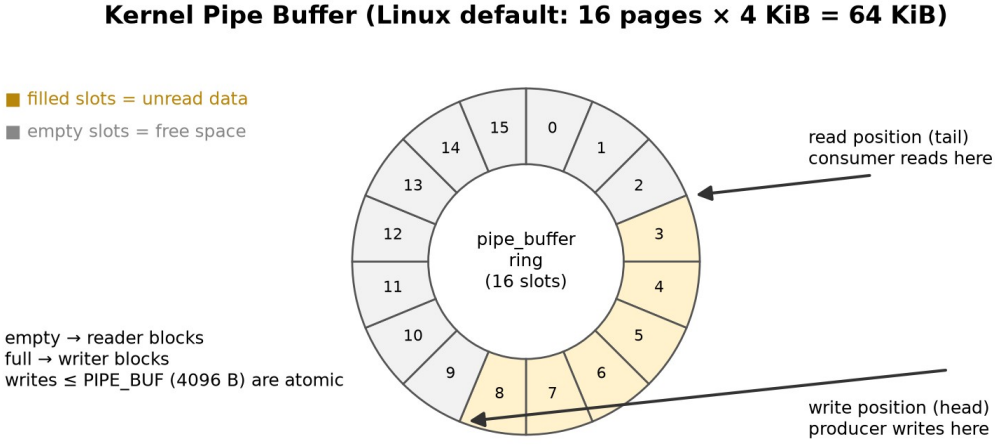


Fig. 4. Structure of the kernel pipe buffer on Linux.

TABLE V. PIPE PARAMETERS MEASURED ON THE TEST SYSTEM

Parameter	Value	How it was obtained
Default pipe capacity	65,536 bytes (64 KiB)	fcntl(fd, F_GETPIPE_SZ) and a non-blocking write loop
PIPE_BUF (atomic write size)	4,096 bytes	PIPE_BUF constant from <limits.h>
Maximum pipe size for users	1,048,576 bytes (1 MiB)	/proc/sys/fs/pipe-max-size
Behaviour when buffer is full	Writer blocks	Blocking write on a full pipe
Behaviour when buffer is empty	Reader blocks; EOF once all write ends are closed	read() returns 0

Two rules follow from this structure. First, a pipe has a finite capacity: if the producer writes faster than the consumer reads, the producer is put to sleep until space becomes free. This is why a pipeline can move gigabytes of data while using only 64 KiB of kernel memory. Second, POSIX guarantees that a write of at most PIPE_BUF bytes (4,096 on Linux) is atomic, meaning it will not be interleaved with data from another writer. Larger writes may be split. The Command Piping Tool has a single writer, so atomicity is not a concern, but the capacity rule is visible in Test T7, where 1 MiB, sixteen times the buffer size, passes through the pipe correctly.

f. Process Life Cycle

Each child created by the tool passes through the states shown in Fig. 5. After fork() it is ready to run; once scheduled it runs, calls execvp() and becomes the requested command. Whenever it reads an empty pipe or writes to a full one, the kernel moves it to the blocked state until the other side acts. When the command finishes, the process becomes a zombie: it no longer runs, but its exit status is kept by the kernel until the parent collects it. The parent's call to waitpid() reaps the zombie and frees its process-table entry. If the parent did not wait, the zombies would remain until the parent itself exited.

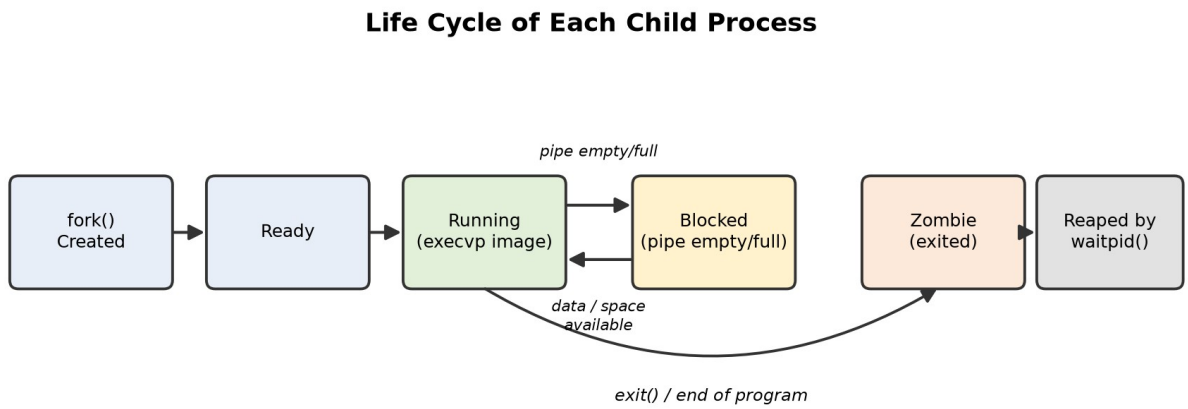


Fig. 5. Life cycle of each child process, including blocking on the pipe and reaping.

g. Algorithm

The complete working of the tool is described by the following steps, by the pseudocode in Listing 1 and by the flowchart in Fig. 6.

Step 1: Read Command 1 and Command 2 from the user and split each line into an argument vector (argv[]), honouring single and double quotes.

Step 2: Call pipe(pfd) to create the anonymous pipe. If it fails, print the error and exit.

Step 3: Call fork() to create Child 1. In Child 1, close the read end, call dup2(pfd[1], STDOUT_FILENO), close the original write end and call execvp() on Command 1.

Step 4: Call fork() again to create Child 2. In Child 2, close the write end, call dup2(pfd[0], STDIN_FILENO), close the original read end and call execvp() on Command 2.

Step 5: If execvp() returns in either child, the command could not be run: print the error with perror() and terminate the child with exit status 127.

Step 6: In the parent, close both ends of the pipe so that only the children hold them.

Step 7: Call waitpid() for Child 1 and Child 2 and report whether each exited normally or was terminated by a signal.

Step 8: Command 2 writes the final output to the terminal; the program ends.

Listing 1: Pseudocode of the Command Piping Tool

```
ALGORITHM CommandPipingTool
INPUT : line1, line2   (two command lines typed by the user)
OUTPUT: output of Command 2 on the terminal, status of both children

 1  cmd1 <- PARSE(line1);  cmd2 <- PARSE(line2)
 2  IF cmd1 is empty OR cmd2 is empty THEN report error; STOP
 3  IF pipe(pfd) fails THEN report error; STOP
 4  c1 <- fork()
 5  IF c1 = 0 THEN                          // Child 1 (producer)
 6      close(pfd[0]); dup2(pfd[1], 1); close(pfd[1])
 7      execvp(cmd1);  report error; _exit(127)
 8  c2 <- fork()
 9  IF c2 = 0 THEN                          // Child 2 (consumer)
10      close(pfd[1]); dup2(pfd[0], 0); close(pfd[0])
11      execvp(cmd2);  report error; _exit(127)
12  close(pfd[0]); close(pfd[1])           // parent keeps no pipe end
13  waitpid(c1, s1); waitpid(c2, s2)
14  REPORT(s1); REPORT(s2)                 // exit code or signal
```

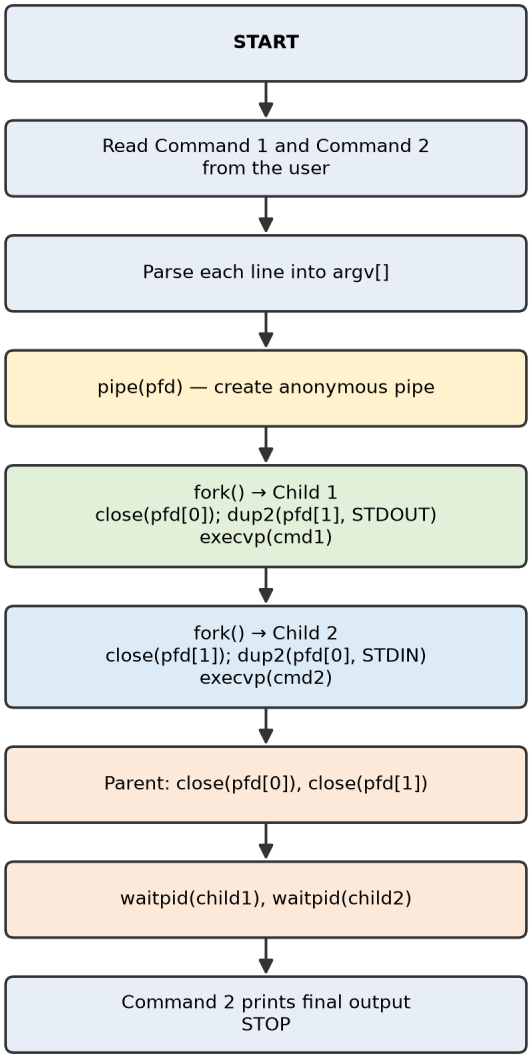


Fig. 6. Flowchart of the complete implementation, from reading the two commands to waiting for both children.

h. Command Parsing

execvp() expects the command as an array of strings ending with NULL, for example {"tr", "a-z", "A-Z", NULL}. The function parse_command() in Listing 2 converts the line typed by the user into this form without allocating memory: it walks through the line, skips blanks, marks the start of each word, and writes a terminating null character after it. When a word begins with a single or double quote, everything up to the matching quote is kept as one argument, so echo "hello operating system" passes a single argument to echo. Backslash sequences such as \n are passed through unchanged and interpreted by the command itself, as Test T8 shows.

Listing 2: The parse_command() function

```
static int parse_command(char *line, char *argv[])
{
    int argc = 0;
    char *p = line;
    while (*p && argc < MAX_ARGS - 1) {
        while (*p == ' ' || *p == '\t' || *p == '\n') p++;   /* skip blanks */
        if (!*p) break;
        if (*p == '"' || *p == '\'') {                       /* quoted word */
            char q = *p++;
            argv[argc++] = p;
            while (*p && *p != q) p++;
```

```
        } else {                                            /* plain word  */
            argv[argc++] = p;
            while (*p && *p != ' ' && *p != '\t' && *p != '\n') p++;
        }
        if (*p) *p++ = '\0';                                /* terminate   */
    }
    argv[argc] = NULL;                                      /* for execvp  */
    return argc;
}
```

i. I/O Redirection and File-Descriptor Logic

Every process starts with three standard descriptors: 0 for standard input, 1 for standard output and 2 for standard error. After pipe() and fork(), each child also holds descriptors 3 and 4 for the two ends of the pipe. Fig. 7 shows how the descriptor table of each child changes. In Child 1, dup2(pfd[1], 1) makes descriptor 1 point to the write end, so anything Command 1 prints goes into the pipe. In Child 2, dup2(pfd[0], 0) makes descriptor 0 point to the read end, so Command 2 reads the pipe as if it were the keyboard. The original descriptors 3 and 4 are then closed because they are no longer needed. Standard error is left untouched in both children so that error messages still appear on the terminal.

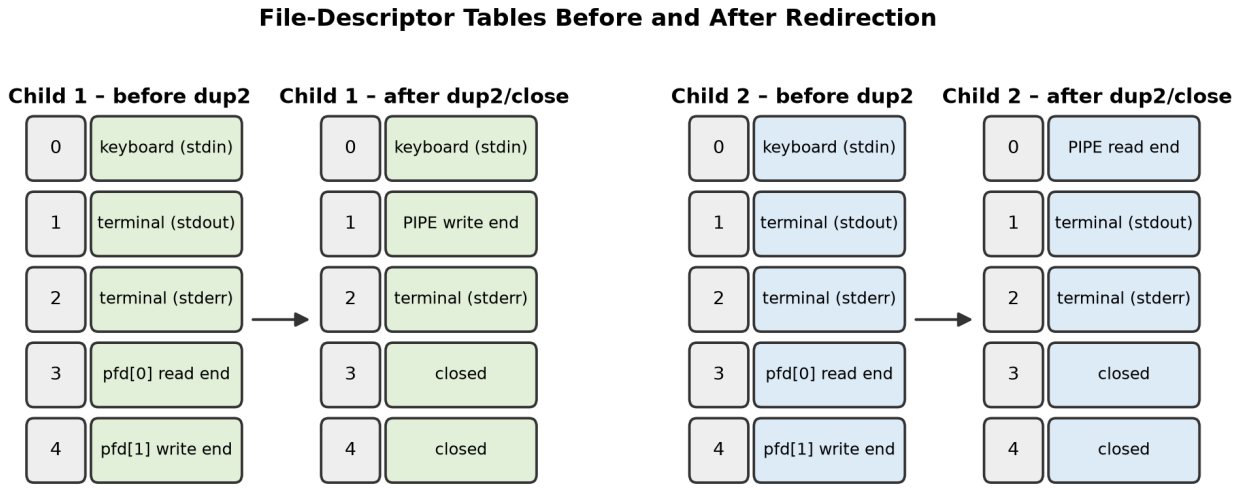


Fig. 7. File-descriptor tables of Child 1 and Child 2 before and after dup2() and close().

The core of the implementation that performs this redirection is shown in Listing 3. The full program is given in Section 4.

Listing 3: Core pipe, fork, dup2 and execvp logic

```
int pfd[2];
if (pipe(pfd) == -1) { perror("pipe"); exit(EXIT_FAILURE); }

pid_t c1 = fork();
if (c1 == 0) {                          /* Child 1: producer   */
    close(pfd[0]);                      /* unused read end     */
    dup2(pfd[1], STDOUT_FILENO);        /* stdout -> pipe      */
    close(pfd[1]);
    execvp(cmd1[0], cmd1);
    perror("execvp (command 1)"); _exit(127);
}

pid_t c2 = fork();
if (c2 == 0) {                          /* Child 2: consumer   */
    close(pfd[1]);                      /* unused write end    */
    dup2(pfd[0], STDIN_FILENO);         /* stdin <- pipe       */
    close(pfd[0]);
    execvp(cmd2[0], cmd2);
    perror("execvp (command 2)"); _exit(127);
}
```

```
close(pfd[0]); close(pfd[1]);          /* parent closes both  */
waitpid(c1, &status1, 0);
waitpid(c2, &status2, 0);
```

j. Process Synchronisation and Correctness

The pipe itself provides synchronisation between the producer and the consumer. If the consumer tries to read from an empty pipe, the kernel blocks it until data arrives. If the producer fills the pipe buffer, the kernel blocks it until the consumer reads. When every write end of the pipe has been closed and the buffer is empty, read() returns 0, which programs interpret as end-of-file (EOF). Fig. 8 shows the order of these events.

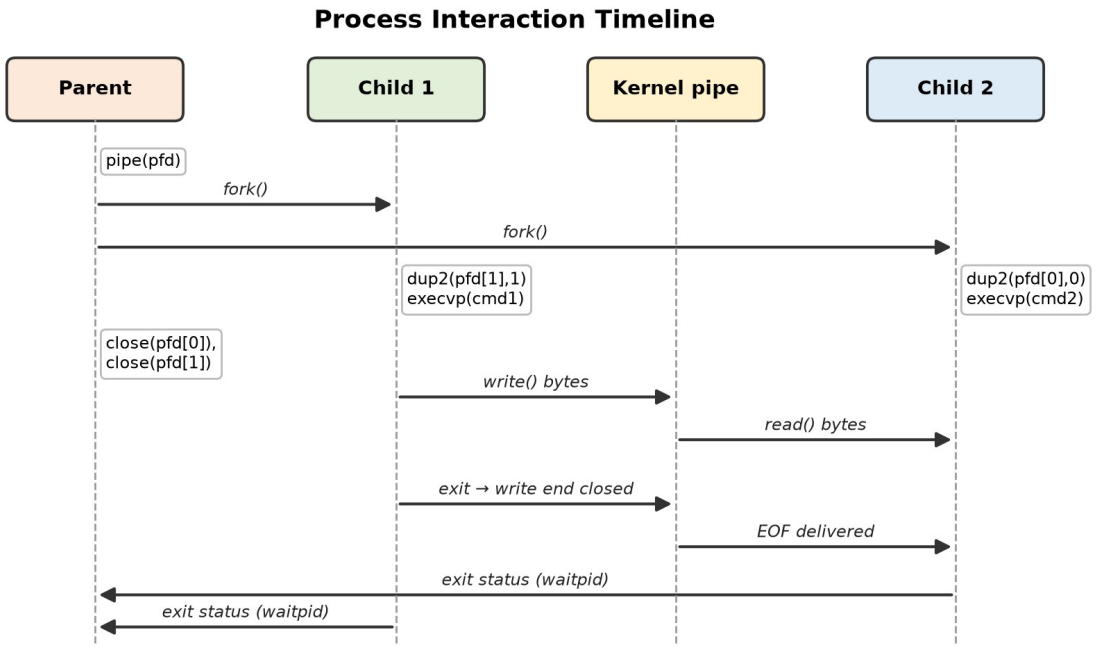


Fig. 8. Process interaction timeline between the parent, the two children and the kernel pipe buffer.

This EOF rule is the reason why closing unused descriptors is essential. If the parent, or Child 2 itself, kept a copy of the write end open, the kernel would consider the pipe still writable and Command 2 (for example wc -l) would wait for more input forever, as Fig. 9 illustrates. The reverse case matters too: if the consumer exits while the producer is still writing, the kernel sends SIGPIPE to the producer, whose default action is to terminate it. This is why Child 1 closes the read end; otherwise it would hold a reader open on its own pipe and block on a full buffer instead of receiving SIGPIPE. Finally, the parent calls waitpid() on both children, which prevents zombie processes and allows the tool to report how each command finished.

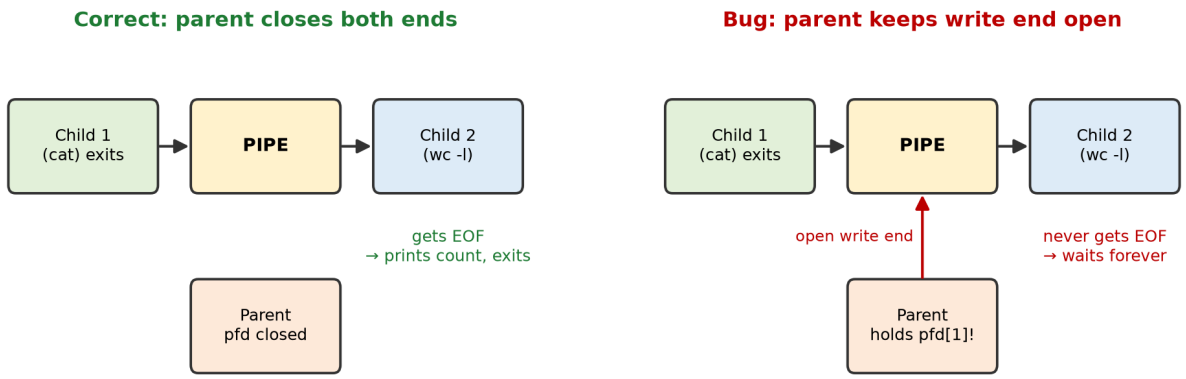


Fig. 9. Why every unused write end must be closed: correct design versus a descriptor leak.

k. Error Handling and Status Reporting

Each system call is checked for failure. If pipe() or fork() fails, the parent prints a descriptive message with perror() and exits with EXIT_FAILURE. If execvp() fails because the command does not exist or is not executable, it returns -1; the child then prints an error and calls _exit(127), the same status that shells use for "command not found". _exit() is used instead of exit() in the child so that buffered standard output is not flushed a second time. Empty input for either command is rejected before any process is created.

The status returned by waitpid() encodes either a normal exit code or the number of the signal that killed the process. The function report() in Listing 4 decodes it with the POSIX macros WIFEXITED(), WEXITSTATUS(), WIFSIGNALED() and WTERMSIG(), and uses strsignal() to print a readable signal name.

Listing 4: Decoding the status of each child

```
static void report(const char *name, pid_t pid, int status)
{
    if (WIFEXITED(status))
        printf("[parent] %s (pid %d) exited with status %d\n",
               name, (int)pid, WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[parent] %s (pid %d) terminated by signal %d (%s)\n",
               name, (int)pid, WTERMSIG(status), strsignal(WTERMSIG(status)));
}
```

4. Experiments

The experiments were designed to confirm that the output of Command 1 is correctly delivered to the input of Command 2, that the parent waits for both children, that failures are reported correctly, and that the tool performs on par with the shell.

a. Experimental Setup

The tool was developed and tested in the environment summarised in Table VI. The program is a single C source file and requires no external libraries beyond the standard C library.

TABLE VI. HARDWARE AND SOFTWARE ENVIRONMENT

Item	Specification
Operating system	Linux (Ubuntu / any POSIX-compliant distribution); kernel 6.x
Programming language	C (C99 / C11)
Compiler	GCC with the -Wall flag (Clang also supported)
Interface	Unix/Linux terminal (Bash)
Core system calls	pipe(), fork(), dup2(), close(), execvp(), waitpid()
Hardware	Any x86-64 or ARM64 machine; tests ran on a 2-core system
Source file	pipetool.c (single file, about 120 lines)

The program is compiled and run with the commands in Listing 5. The -Wall flag enables all common compiler warnings; the program compiles without any warning.

Listing 5: Compiling and running the tool

```
gcc -Wall -o pipetool pipetool.c
./pipetool
```

b. Implementation

The complete source code of the tool is given in Listing 6. The function parse_command() converts a line of text into the NULL-terminated argument array expected by execvp(); read_line() prints a prompt and reads one line; report() decodes the status of a child; and main() performs the pipeline, printing each step so that the user can follow what the operating system is doing.

Listing 6: Complete source code (pipetool.c)

```
/*
 * Command Piping Tool
 * Connects the STDOUT of Command 1 to the STDIN of Command 2
 * using pipe(), fork(), dup2(), execvp() and waitpid().
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_LINE 1024
#define MAX_ARGS 64

/* Split a command line into an argv[] array (handles "quoted words"). */
static int parse_command(char *line, char *argv[])
{
    int argc = 0;
    char *p = line;
    while (*p && argc < MAX_ARGS - 1) {
        while (*p == ' ' || *p == '\t' || *p == '\n') p++;
        if (!*p) break;
        if (*p == '"' || *p == '\'') {
            char q = *p++;
            argv[argc++] = p;
            while (*p && *p != q) p++;
        } else {
            argv[argc++] = p;
            while (*p && *p != ' ' && *p != '\t' && *p != '\n') p++;
        }
        if (*p) *p++ = '\0';
    }
    argv[argc] = NULL;
    return argc;
```

```
}

static void read_line(const char *prompt, char *buf)
{
    printf("%s", prompt);
    fflush(stdout);
    if (!fgets(buf, MAX_LINE, stdin)) {
        fprintf(stderr, "Input error\n");
        exit(EXIT_FAILURE);
    }
    buf[strcspn(buf, "\n")] = '\0';
}

/* Print how a child finished: normal exit code or terminating signal. */
static void report(const char *name, pid_t pid, int status)
{
    if (WIFEXITED(status))
        printf("[parent] %s (pid %d) exited with status %d\n",
               name, (int)pid, WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[parent] %s (pid %d) terminated by signal %d (%s)\n",
               name, (int)pid, WTERMSIG(status), strsignal(WTERMSIG(status)));
}

int main(void)
{
    char line1[MAX_LINE], line2[MAX_LINE];
    char *cmd1[MAX_ARGS], *cmd2[MAX_ARGS];
    int pfd[2];
    pid_t c1, c2;
    int status1, status2;

    printf("===== COMMAND PIPING TOOL =====\n");
    read_line("Command 1: ", line1);
    read_line("Command 2: ", line2);

    if (parse_command(line1, cmd1) == 0 || parse_command(line2, cmd2) == 0) {
        fprintf(stderr, "Error: both commands must be non-empty\n");
        return EXIT_FAILURE;
    }

    printf("Creating anonymous pipe...\n");
    if (pipe(pfd) == -1) { perror("pipe"); return EXIT_FAILURE; }

    printf("Forking child 1...\n");
    printf("Forking child 2...\n");
    printf("Redirecting STDOUT -> pipe\n");
    printf("Redirecting STDIN  <- pipe\n");
    printf("Executing commands...\n\nFinal output:\n");
    fflush(stdout);

    c1 = fork();
    if (c1 == -1) { perror("fork"); return EXIT_FAILURE; }
    if (c1 == 0) {                          /* Child 1: producer */
        close(pfd[0]);                      /* unused read end   */
        if (dup2(pfd[1], STDOUT_FILENO) == -1) { perror("dup2"); _exit(1); }
        close(pfd[1]);
        execvp(cmd1[0], cmd1);
        perror("execvp (command 1)");
        _exit(127);
    }

    c2 = fork();
    if (c2 == -1) { perror("fork"); return EXIT_FAILURE; }
    if (c2 == 0) {                          /* Child 2: consumer */
        close(pfd[1]);                      /* unused write end  */
        if (dup2(pfd[0], STDIN_FILENO) == -1) { perror("dup2"); _exit(1); }
        close(pfd[0]);
        execvp(cmd2[0], cmd2);
        perror("execvp (command 2)");
        _exit(127);
    }
```

```
    /* Parent: close both ends so Command 2 receives EOF */
    close(pfd[0]);
    close(pfd[1]);
    waitpid(c1, &status1, 0);
    waitpid(c2, &status2, 0);

    printf("\n");
    report("child 1", c1, status1);
    report("child 2", c2, status2);
    return EXIT_SUCCESS;
}
```

c. Core Test Cases

Four core test cases were chosen, as listed in Table VII. The first three are the representative pipelines from the project presentation, and each one proves a different property of the pipe. The fourth deliberately uses a command that does not exist.

TABLE VII. CORE TEST CASES AND EXPECTED OUTPUT

Test	Command 1	Command 2	Purpose	Expected output
T1	ls -l	grep .c	Filtering	Only the directory lines that match .c
T2	cat /etc/passwd	wc -l	Counting	A single integer: the number of lines in /etc/passwd
T3	echo "hello operating system"	tr a-z A-Z	Transformation	HELLO OPERATING SYSTEM
T4	lsx -l (invalid)	wc -l	Error handling	execvp error for Command 1; Command 2 prints 0; child 1 exits with 127

- **Filtering test (T1):** proves that the directory listing produced by ls -l is passed to a filtering command and that only matching lines reach the terminal.
- **Counting test (T2):** proves that the second command can consume all of the data written by the first command and only then compute a result, which depends on correct EOF delivery.
- **Transformation test (T3):** proves that the stream received by the second process is complete and can be transformed character by character.
- **Error-handling test (T4):** proves that a failure in one child is detected, reported, and does not hang the other child or the parent.

d. Extended Test Cases

Six further tests, listed in Table VIII, were added to exercise situations that the core tests do not reach: larger data volumes, a quoted argument containing escape sequences, a failure in the consumer rather than the producer, and a consumer that stops reading before the producer has finished.

TABLE VIII. EXTENDED TEST CASES

Test	Command 1	Command 2	Property checked	Expected output
T5	ls	sort -r	Sorting a stream	File names in reverse order
T6	seq 1 100000	tail -n 3	Data larger than one write	99998, 99999, 100000
T7	head -c 1048576 /dev/zero	wc -c	Data larger than the 64 KiB buffer	1048576
T8	printf "banana\napple\ncherry\napple\n"	sort -u	Quoted argument with escapes	apple, banana, cherry
T9	ls -l	grepx foo (invalid)	Error in the consumer	execvp error for Command 2; child 2 exits with 127
T10	seq 1 1000000	head -n 2	Consumer exits early	1, 2; child 1 terminated by SIGPIPE

e. Performance Benchmark

To measure how efficiently data moves through the tool, the pipeline head -c N /dev/zero | wc -c was run through the tool and directly in Bash for N equal to 1, 8, 32, 64, 128 and 256 MB. head produces exactly N bytes and wc -c counts them, so almost all of the work is the transfer through the pipe. Each configuration was run five times and the median wall-clock time, including process creation, was recorded. Throughput was computed as data volume divided by time.

f. Descriptor-Leak Experiment

To demonstrate the importance of closing descriptors, a modified version of the tool, pipetool_bug.c, was built in which the parent does not close the two ends of the pipe. Everything else is identical. The pipeline cat /etc/passwd | wc -l was run through this version under the timeout utility with a limit of five seconds. If the theory is correct, wc -l should never print its result, because the parent still holds a write end and the kernel cannot deliver EOF.

g. Worked Trace of the Transformation Test

To illustrate the data path, consider the pipeline echo "hello operating system" | tr a-z A-Z. Child 1 executes echo, which writes the string followed by a newline to descriptor 1; because of dup2(), these bytes enter the pipe through pfd[1]. Child 2 executes tr, which reads descriptor 0; because of dup2(), it receives the same bytes from pfd[0]. tr converts every lowercase letter to uppercase and writes HELLO OPERATING SYSTEM to the terminal. When echo exits, the last write end is closed, tr receives EOF and also exits. Fig. 10 shows this trace.

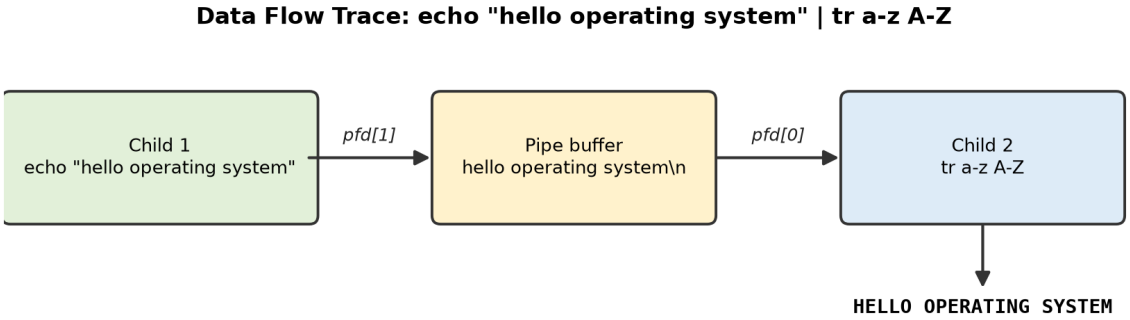


Fig. 10. Data-flow trace of the example pipeline echo "hello operating system" | tr a-z A-Z.

5. RESULTS

This section presents the terminal output of every test, the benchmark measurements and the outcome of the descriptor-leak experiment, followed by a discussion of the results and of the challenges met during the project.

a. Results of the Core Tests

In Test T1 (Fig. 11) the directory contained six files. Only the three lines for main.c, pipetool.c and sample.c reached the terminal, which confirms that the output of ls -l was delivered to grep. Note that grep treats its pattern as a regular expression, so the dot in .c matches any character; a stricter pattern such as '\.c$' matches only names that end in .c.

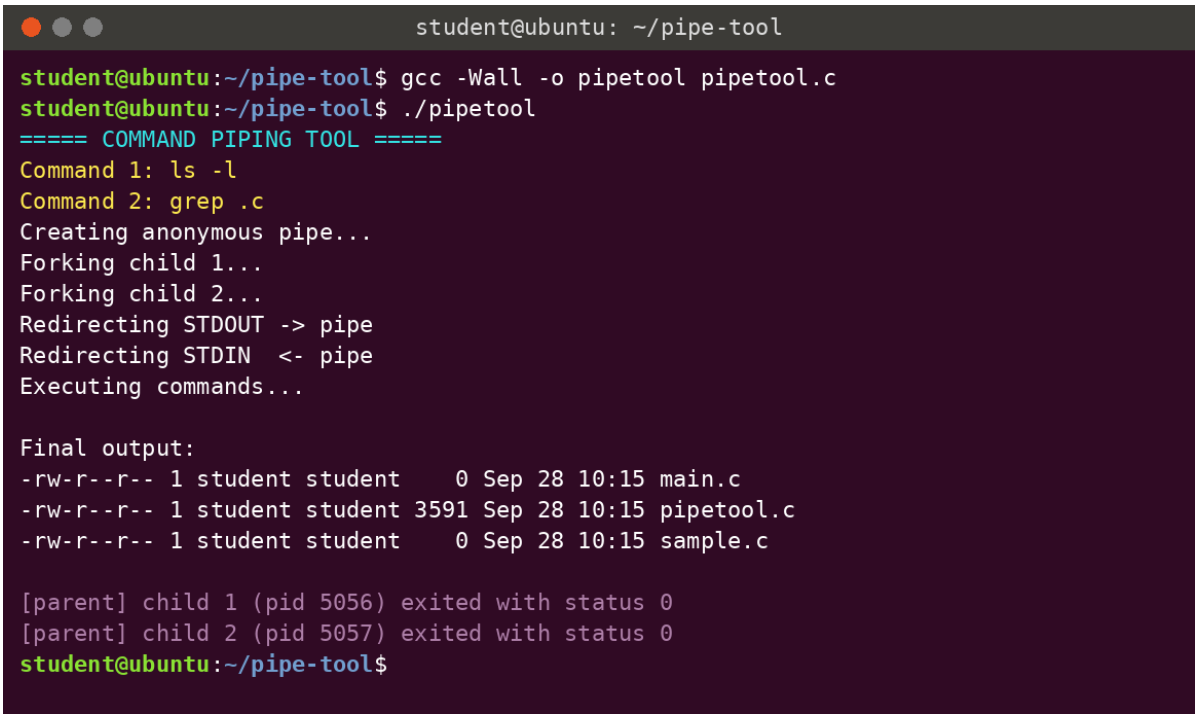


Fig. 11. Compilation of the tool and execution of Test T1 (ls -l | grep .c).

In Test T2 (Fig. 12) the tool printed 19, the number of lines in /etc/passwd on the test machine. The exact number depends on the accounts present on a system; what matters is that it matches the value Bash reports on the same machine. Because wc prints its count only after reading EOF, this result also confirms that all unused write ends were closed.

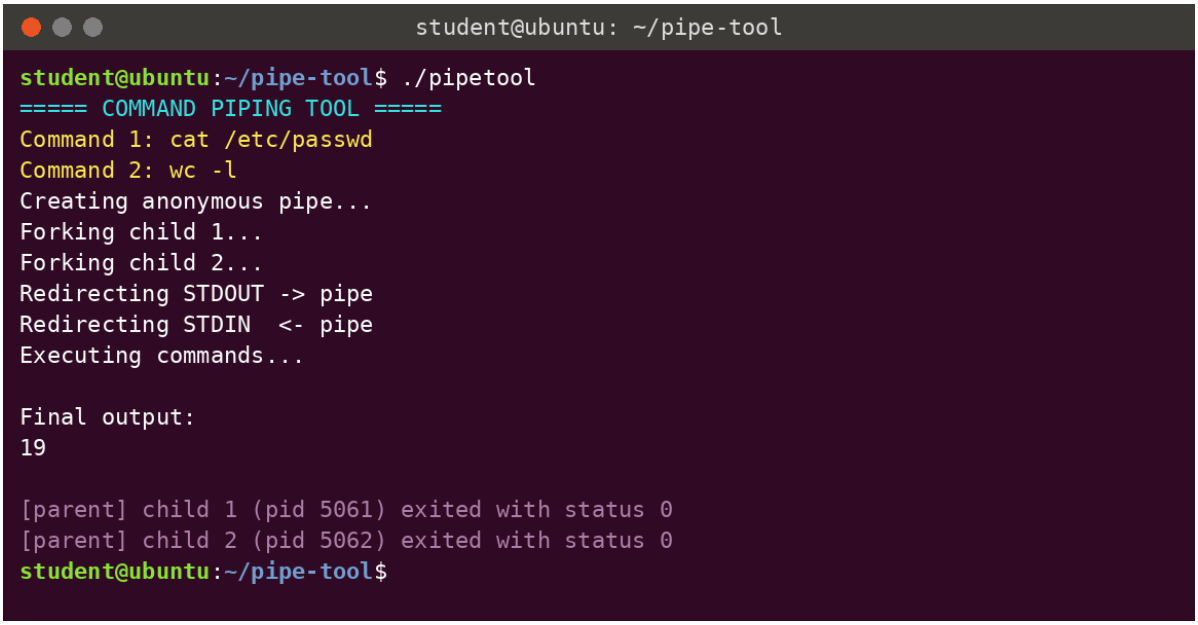


Fig. 12. Execution of Test T2 (cat /etc/passwd | wc -l).

In Test T3 (Fig. 13) the tool printed HELLO OPERATING SYSTEM, exactly as expected, showing that the full text written by echo was transferred through the pipe and transformed by tr.

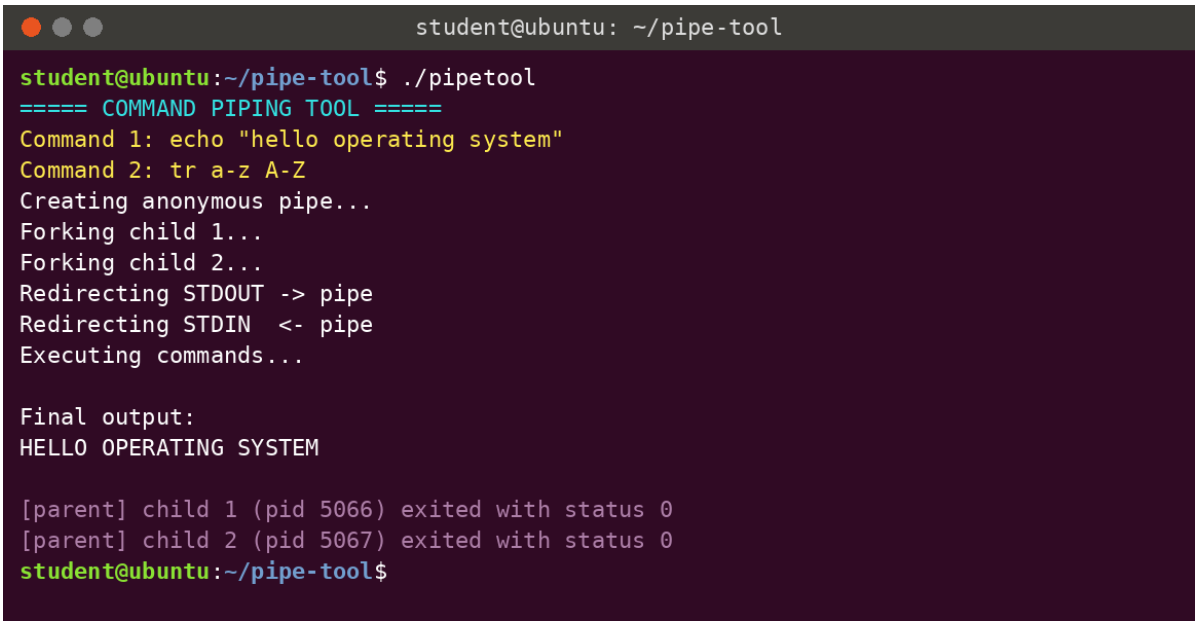


Fig. 13. Execution of Test T3 (echo "hello operating system" | tr a-z A-Z).

In Test T4 (Fig. 14) Command 1 did not exist. execvp() failed, Child 1 printed "No such file or directory" on standard error and exited with status 127. Because the failed child closed its write end on exit, Child 2 (wc -l) immediately received EOF and printed 0. The parent collected both statuses and terminated normally, so no process was left hanging.

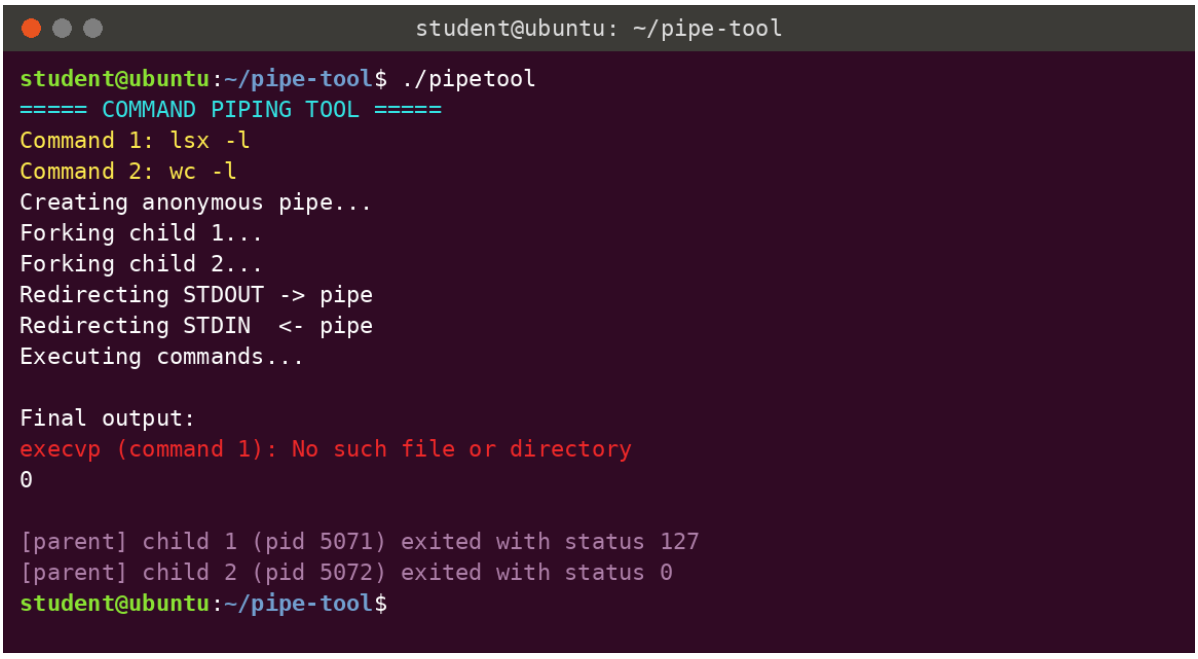


Fig. 14. Execution of Test T4 with an invalid first command (lsx -l | wc -l).

The reference run in Bash (Fig. 15) produced the same output for the three valid pipelines.

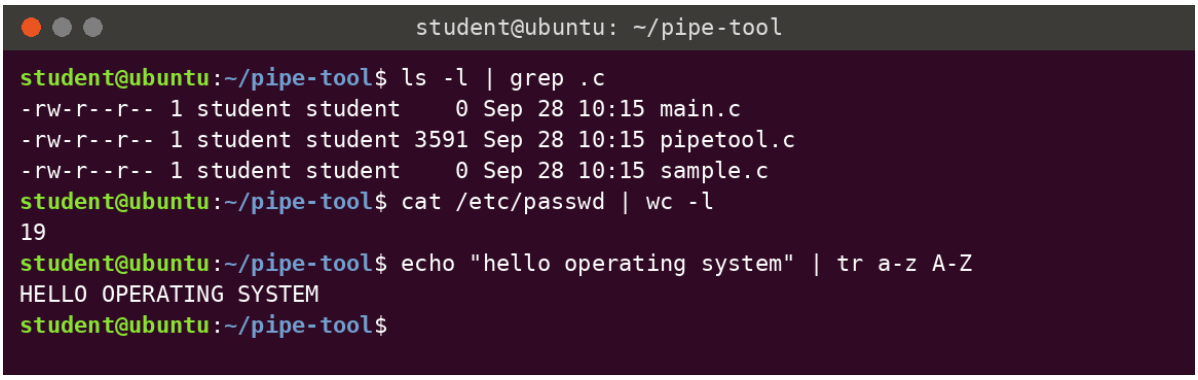


Fig. 15. Reference output of the T1 to T3 pipelines executed directly in the Bash shell.

b. Results of the Extended Tests

Test T5 (Fig. 16) sorted the directory listing in reverse order, confirming that a command which must read its whole input before producing any output works correctly through the pipe.



Fig. 16. Execution of Test T5 (ls | sort -r).

Test T6 (Fig. 17) sent 100,000 numbers, about 575 KB of text, through the pipe; tail printed the last three, 99998, 99999 and 100000. Test T7 (Fig. 18) sent exactly 1,048,576 bytes, sixteen times the pipe capacity, and wc -c counted every byte. Together these results show that the producer was blocked and woken repeatedly as the buffer filled and emptied, with no data lost.



Fig. 17. Execution of Test T6 (seq 1 100000 | tail -n 3).

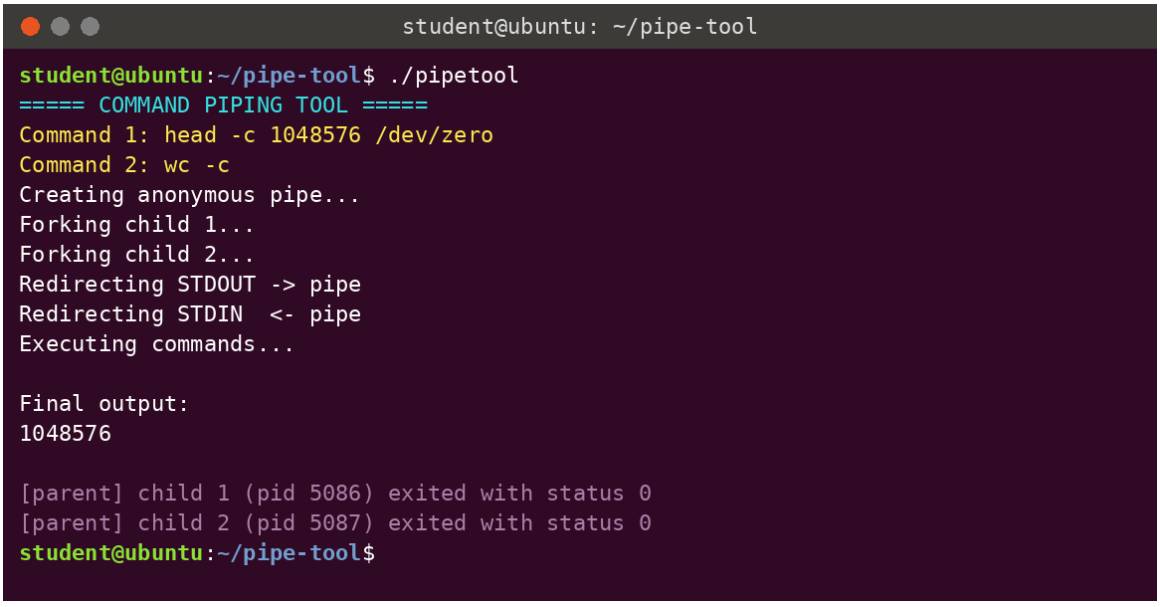


Fig. 18. Execution of Test T7: 1 MiB transferred through the pipe (head -c 1048576 /dev/zero | wc -c).

Test T8 (Fig. 19) passed the quoted argument "banana\napple\ncherry\napple\n" to printf as a single word. printf expanded the escape sequences into four lines and sort -u produced the three unique fruits in order, confirming the quote handling in parse_command().

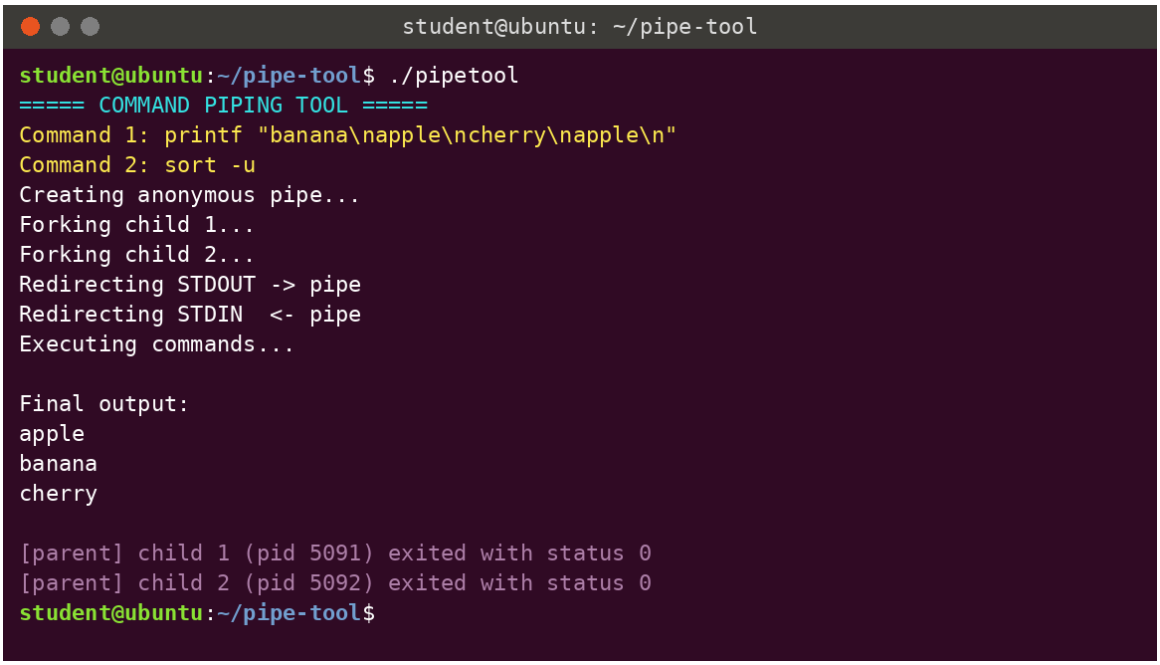


Fig. 19. Execution of Test T8 with a quoted, escaped argument (printf ... | sort -u).

In Test T9 (Fig. 20) the consumer command did not exist. Child 2 reported the execvp error and exited with status 127. Child 1 (ls -l) exited normally with status 0 because its small output fitted entirely in the pipe buffer before the consumer disappeared.

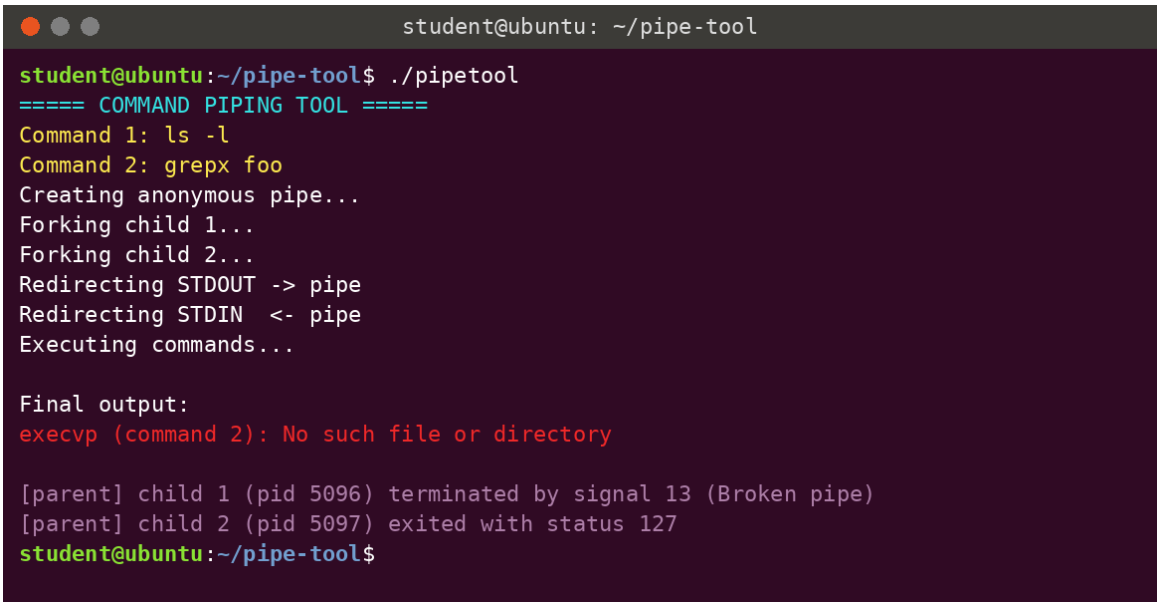


Fig. 20. Execution of Test T9 with an invalid second command (ls -l | grepx foo).

Test T10 (Fig. 21) is the most interesting case. seq 1 1000000 produces about 6.9 MB, but head -n 2 exits after printing two lines. When seq next tries to write, there is no reader left, so the kernel sends it SIGPIPE and it is terminated. The first version of the tool reported this child as having exited with status 0, because it applied WEXITSTATUS() without first checking how the child ended. After the status reporting was corrected (Listing 4), the tool correctly prints that child 1 was terminated by signal 13 (Broken pipe). This is the same behaviour Bash shows, where PIPESTATUS records 141 (128 + 13) for the first stage.

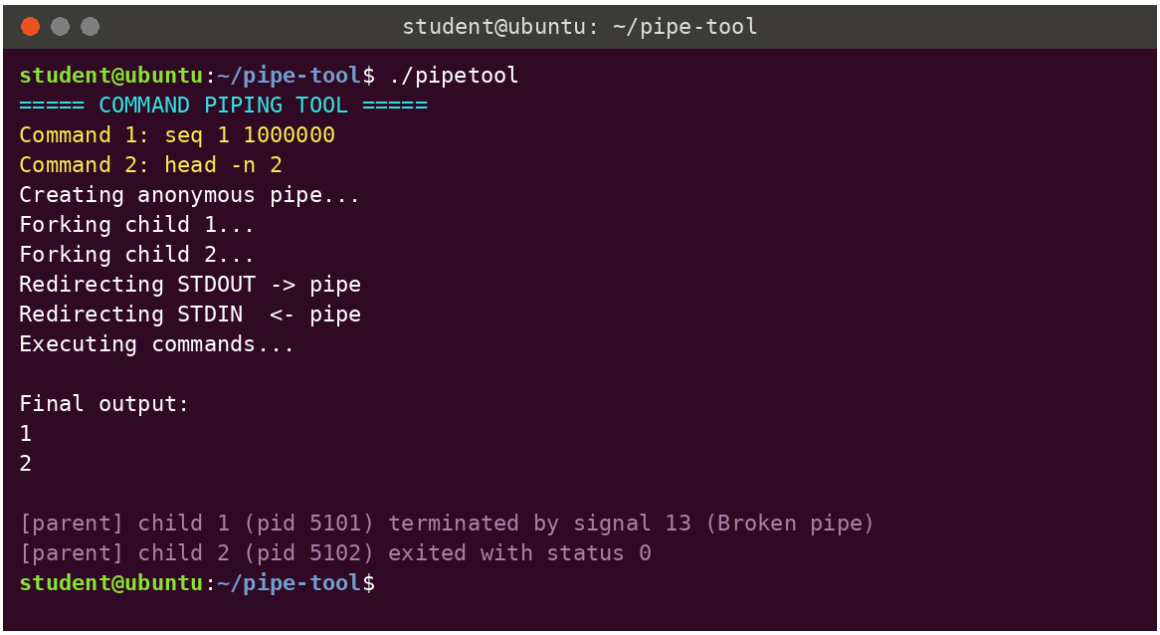


Fig. 21. Execution of Test T10: the consumer exits early and the producer is terminated by SIGPIPE.

Table IX summarises the results of all ten tests. Every test passed, and for every valid pipeline the output of the tool was identical to that of Bash.

TABLE IX. RESULTS OF ALL TEST CASES COMPARED WITH THE BASH SHELL

Test	Output of the tool	Output of Bash	Child 1 / Child 2	Result
T1	main.c, pipetool.c, sample.c lines	Identical	0 / 0	**Pass**
T2	19	19	0 / 0	**Pass**
T3	HELLO OPERATING SYSTEM	HELLO OPERATING SYSTEM	0 / 0	**Pass**
T4	execvp error, then 0	command not found, then 0	127 / 0	**Pass**
T5	utils.h ... Makefile (reverse)	Identical	0 / 0	**Pass**
T6	99998 99999 100000	Identical	0 / 0	**Pass**
T7	1048576	1048576	0 / 0	**Pass**
T8	apple banana cherry	Identical	0 / 0	**Pass**
T9	execvp error for Command 2	command not found	0 / 127	**Pass**
T10	1 2	1 2	SIGPIPE / 0	**Pass**

c. *Performance Results*

The benchmark results are listed in Table X and plotted in Fig. 22 and Fig. 23. For small volumes the cost of creating the processes dominates, so throughput is low for both the tool and Bash. As the volume grows, throughput rises and levels off at about 2.6 GB/s, where it is limited by the speed at which the kernel can copy data through the 64 KiB buffer. The execution time grows linearly with the data volume, as expected. From 32 MB upwards the tool and Bash were within about 5% of each other, which is natural because both use the same kernel mechanism. For 1 MB the tool was faster (1.7 ms against 2.2 ms) because Bash must first start a shell process before it can create the pipeline.

TABLE X. BENCHMARK RESULTS (MEDIAN OF FIVE RUNS)

Data volume	Tool time (ms)	Bash time (ms)	Tool throughput (MB/s)	Bash throughput (MB/s)
1 MB	1.7	2.2	574	448
8 MB	4.4	4.9	1,828	1,649
32 MB	13.1	13.7	2,449	2,331
64 MB	25.5	25.4	2,509	2,517
128 MB	48.6	48.4	2,633	2,645
256 MB	96.4	96.6	2,655	2,651

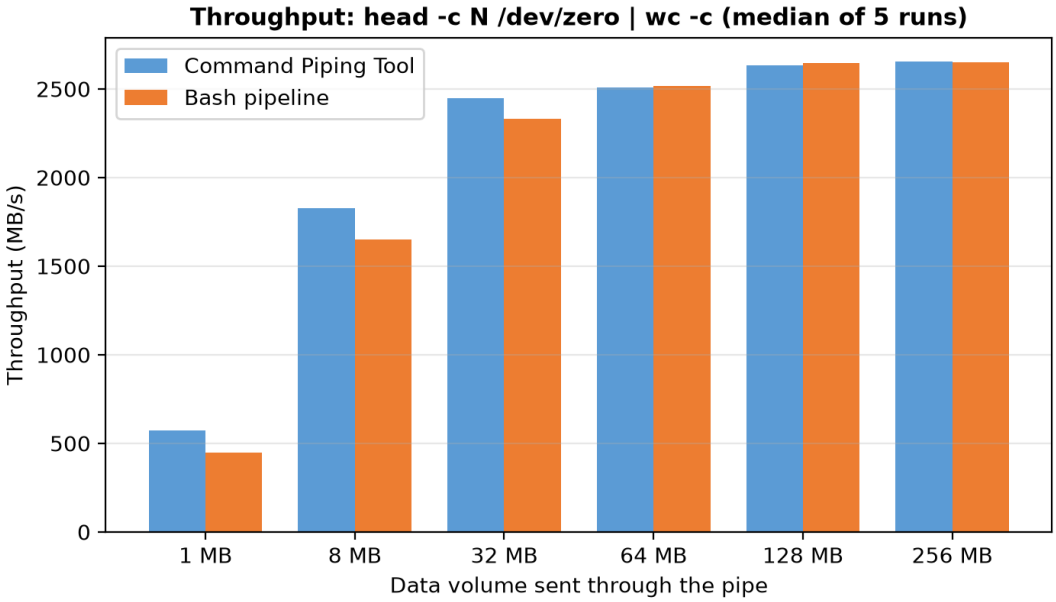


Fig. 22. Throughput of the tool and of Bash for data volumes from 1 MB to 256 MB.

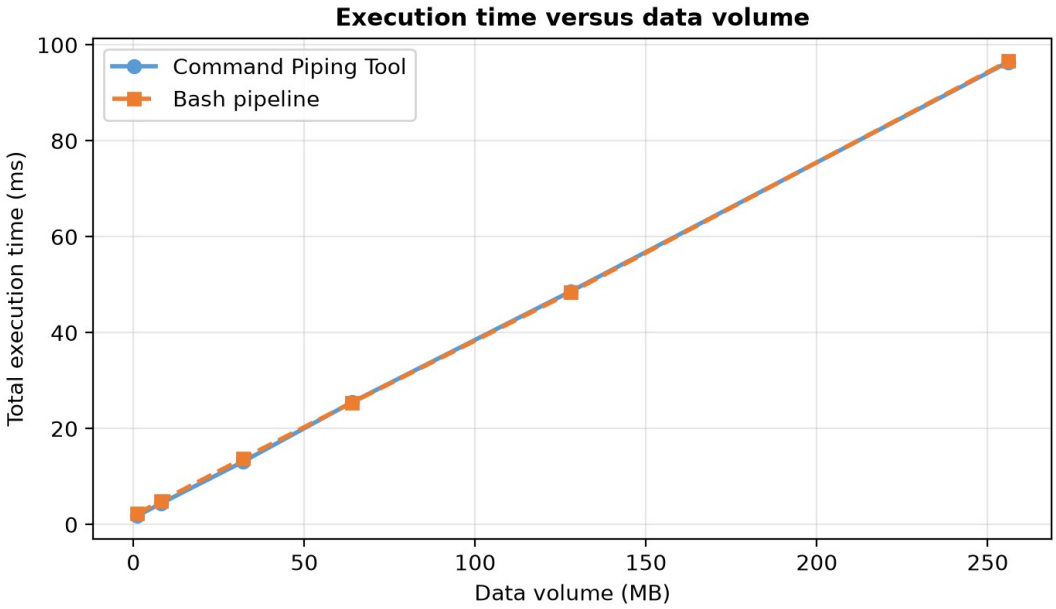


Fig. 23. Total execution time versus data volume.

d. Result of the Descriptor-Leak Experiment

The output of the modified program is shown in Fig. 24. The steps were printed as usual, but no result ever appeared under Final output. After five seconds timeout terminated the program and reported exit status 124, which means the time limit was reached.

The correct version completes the same pipeline instantly (Fig. 12). This confirms experimentally that a single forgotten close() of a write end is enough to make the consumer wait forever, exactly as predicted in Fig. 9.

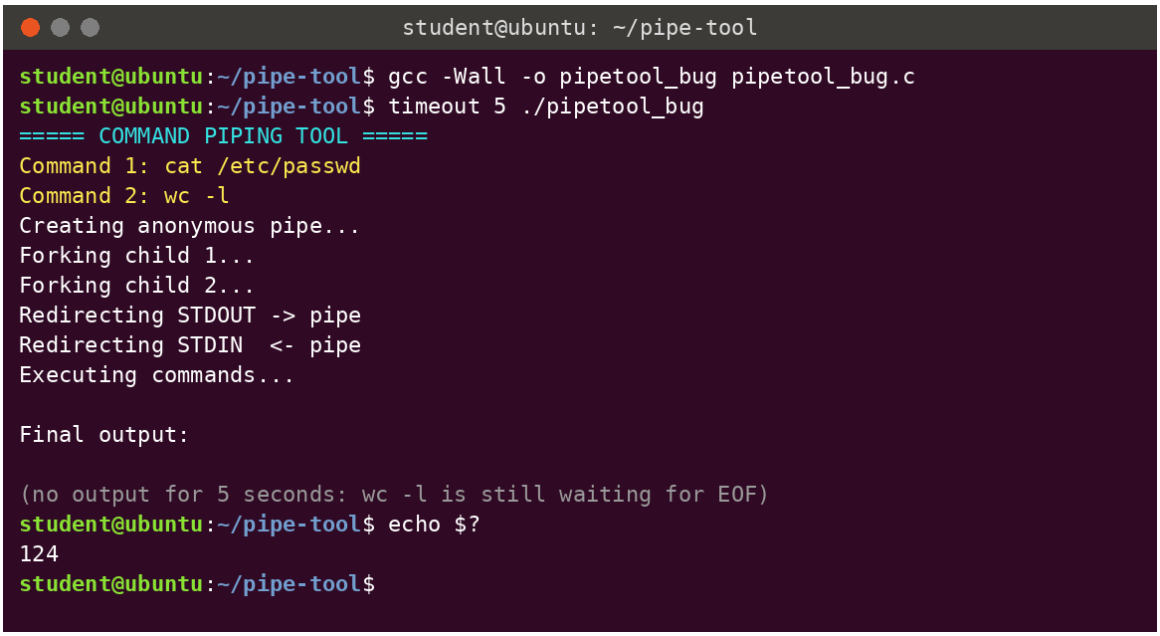


Fig. 24. Descriptor-leak experiment: the parent does not close the pipe, and wc -l waits until killed by timeout.

e. Discussion

The results satisfy every functional and non-functional requirement. The validation checklist from the design stage was met in all valid runs: the pipe was created, two children were forked, standard output and standard input were redirected, both commands were executed and the parent waited for both children. The tests also show the three ways a pipeline can end: normally when the producer finishes and the consumer reads EOF (T1 to T8), with an error when a command cannot be started (T4 and T9), and with SIGPIPE when the consumer stops early (T10). Handling all three correctly is what separates a working pipeline from a fragile one. Table XI compares the capabilities of the tool with those of the Bash shell.

TABLE XI. COMPARISON OF THE COMMAND PIPING TOOL WITH THE BASH SHELL PIPELINE

Feature	Command Piping Tool	Bash shell
Number of pipeline stages	Two	Any number
Input method	Two prompted lines	Single command line with |
Quoted arguments	Supported (single and double quotes)	Fully supported, with escapes
I/O redirection (<, >)	Not supported	Supported
Environment variable expansion	Not supported	Supported
Visibility of internal steps	Each step printed for learning	Hidden from the user
Exit status reporting	Both children, including signals	Last stage ($?) or PIPESTATUS

f. Challenges Faced

Several problems were met during development; Table XII lists them with their causes and the solutions adopted. Most of them relate to the same theme: in a multi-process program, every process holds its own copy of shared resources such as descriptors and output buffers, and each copy must be managed explicitly.

TABLE XII. CHALLENGES FACED AND SOLUTIONS ADOPTED

Challenge	Cause	Solution
Command 2 (wc -l) never finished	A write end of the pipe was still open in another process	Close unused ends in both children and both ends in the parent
Step messages printed twice	stdout buffer copied into children by fork()	fflush(stdout) before fork() and _exit() in children
Quoted text split into words	Simple space-based tokenising	parse_command() keeps quoted text together
Child killed by SIGPIPE shown as status 0	WEXITSTATUS() used without checking how the child ended	Decode status with WIFEXITED() and WIFSIGNALED()
Error messages hidden in pipe	stderr assumed to follow stdout	Leave descriptor 2 untouched so errors reach the terminal
grep .c matched unexpected lines	The dot is a regular-expression wildcard	Documented; use '\.c$' for an exact suffix match

g. Learning Outcomes

The project gave the team practical experience with the core topics of the Operating Systems and System Programming course. Table XIII maps each concept to the part of the project where it was applied.

TABLE XIII. LEARNING OUTCOMES MAPPED TO OPERATING-SYSTEM CONCEPTS

Concept	Where it appears in the project
Process creation	fork() creates the producer and consumer children
Program execution	execvp() replaces each child with the requested command
Inter-process communication	Anonymous pipe carries bytes from Child 1 to Child 2
File-descriptor management	dup2() and close() rewire standard input and output
Synchronisation	Blocking read/write on the pipe and waitpid() in the parent
Signals	SIGPIPE delivered to the producer when the consumer exits early
Process termination	Exit status, zombies and reaping with waitpid()

6. CONCLUSION AND FUTURE WORK

This project implemented a working Command Piping Tool in C that reproduces the behaviour of a two-stage shell pipeline. Using only the system calls pipe(), fork(), dup2(), close(), execvp() and waitpid(), the tool connects the standard output of one Linux command to the standard input of another and displays the final result. Ten tests covering filtering, counting, sorting, transformation, large data, quoted arguments, invalid commands and early termination all passed, with output identical to Bash. The benchmark showed throughput of about 2.6 GB/s for large transfers, on par with Bash, and the descriptor-leak experiment proved that a single unclosed write end makes the pipeline hang.

The project also showed the value of testing beyond the obvious cases. The SIGPIPE test exposed a real defect in the first version, where a child killed by a signal was reported as a normal exit, and the fix made the tool's status reporting more accurate than a simple exit-code check. The most important lesson is that closing unused descriptors is not a matter of tidiness but a condition for correctness, because end-of-file can only be delivered when every write end is closed.

Beyond its educational value, the techniques used in the tool are the same ones found in shells, build systems, compilers that call external tools, and programs that capture the output of other programs. The tool can be extended in several directions in the future:

- Support for pipelines of any length (cmd1 | cmd2 | ... | cmdN) by creating N-1 pipes and N children in a loop.
- Input and output redirection with <, > and >>, and redirection of standard error with 2>.
- A single-line interface that parses the | symbol directly, together with a repeating prompt so that the tool behaves like a mini shell.
- Built-in commands such as cd and exit, environment-variable expansion, and background execution with &.
- Signal handling for Ctrl+C and job control by placing all children in one process group.
- Use of named pipes (FIFOs) to connect unrelated processes, and a comparison with shared memory for large transfers.
- Reporting of per-stage timing and the number of bytes transferred, to make the tool a small pipeline profiler.

REFERENCES

[1] W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed., Addison-Wesley Professional (2013).

[2] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press (2010).

[3] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, 10th ed., John Wiley & Sons (2018).

[4] Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, 4th ed., Pearson (2015).

[5] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books (2018).

[6] Dennis M. Ritchie and Ken Thompson, "The UNIX Time-Sharing System," Communications of the ACM, 17(7), 365-375 (1974).

[7] M. D. McIlroy, E. N. Pinson, and B. A. Tague, "UNIX Time-Sharing System: Foreword," The Bell System Technical Journal, 57(6), 1899-1904 (1978).

[8] IEEE and The Open Group, "IEEE Std 1003.1-2017 (POSIX.1-2017): pipe, fork, dup2, exec and waitpid," The Open Group Base Specifications Issue 7 (2018).

[9] Linux man-pages project, "pipe(2), fork(2), dup(2), execvp(3), wait(2), pipe(7) and signal(7)," https://man7.org/linux/man-pages/ (accessed 2026).

[10] Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall (1988).

[11] Russ Cox, Frans Kaashoek, and Robert Morris, *xv6: a simple, Unix-like teaching operating system*, MIT PDOS (2020).

[12] Daniel P. Bovet and Marco Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media (2005).

[13] Robert Love, *Linux Kernel Development*, 3rd ed., Addison-Wesley Professional (2010).

[14] Brian W. Kernighan and Rob Pike, *The UNIX Programming Environment*, Prentice Hall (1984).